Prueba de desempeño – Ruta de IA y Automatización

Orquestación de datos con Airflow y plataforma de consumo en tiempo real

Caso de uso

SuperApp es una compañía que está lanzando **SismoWatch**, su nueva vertical de monitoreo sísmico global en tiempo casi real. Las áreas de operaciones y de gestión de riesgo necesitan saber qué está ocurriendo (dónde, cuándo y con qué magnitud) para priorizar decisiones. Hoy esa información solo existe como un feed público que cambia cada minuto: no guarda historial, no se valida, no se resume y nadie puede consultarla de forma cómoda.

Fuiste contratado(a) como **Ingeniero(a) de Datos y Automatizador(a) con IA** para convertir esa fuente pública en un producto de datos confiable: automatizado, seguro, con calidad controlada, con IA aplicada donde realmente aporta valor y con una plataforma donde el equipo pueda ver sus datos en dashboard, en reportes y en tiempo real.

No se evalúa solo que “funcione”: se evalúa que tengas **claro el propósito de la arquitectura que montas** y que puedas defender cada decisión técnica.

Objetivo

Bajo tu rol como Ingeniero(a) de Datos y Automatizador(a) con IA, deberás diseñar e implementar de punta a punta una solución que:

- Se conecte a una **API pública** y extraiga datos de forma automática y repetible.
- Monte un **orquestamiento en Apache Airflow** que procese los datos sin intervención manual.
- Aplique **controles de calidad de datos** y **medidas de seguridad** en todo el flujo.
- Use **IA para automatizar** tareas de análisis y reporte, con controles para que no invente información.
- Entregue una **plataforma propia de consumo de datos** donde el usuario vea sus datos en **dashboard**, en **reportes** y en **tiempo real**.
- Esté respaldada por un documento de arquitectura que explique el **propósito**, las capas y las decisiones tomadas.

Fuente de datos (API pública)

Todos los coders usan la misma fuente para que las soluciones sean comparables: **USGS Earthquake Catalog**, el servicio público del Servicio Geológico de Estados Unidos. **No requiere API key ni registro.**

Uso	URL
Tiempo real (feed GeoJSON)	`https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson`

Uso	URL
Histórico / consultas	`https://earthquake.usgs.gov/fdsnws/event/1/query?format=geojson&starttime=YYYY-MM-DD&endtime=YYYY-MM-DD&minmagnitude=2.5`
Documentación	`https://earthquake.usgs.gov/fdsnws/event/1/`

Campos principales de cada evento (GeoJSON): `id`, `properties.mag`, `properties.place`, `properties.time` (milisegundos epoch), `properties.updated`, `properties.tsunami`, `properties.sig`, `properties.status`, `properties.magType`, `properties.type`, `properties.url` y `geometry.coordinates` = [longitud, latitud, profundidad_km].

Prueba rápida de la fuente desde la terminal:

```
curl -s "https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson" | head -c 600
```

Ten presente que un evento puede **actualizarse** después de publicado (cambia su magnitud o su estado), por lo que tu carga debe poder insertar y actualizar sin duplicar.

Modelo de roles de la plataforma

Este punto condiciona la parte de seguridad: léelo con cuidado antes de empezar.

	Visitante (sin login)	Viewer	Admin
Ver dashboard (KPIs, gráficos, mapa)	No	Sí	Sí
Ver y exportar reportes (CSV / PDF)	No	Sí	Sí
Ver eventos en tiempo real	No	Sí	Sí
Ver estado del pipeline y resumen de calidad	No	Sí (resumen)	Sí (detalle)
Ver registros en cuarentena de calidad	No	No	Sí
Regenerar reporte con IA / lanzar backfill	No	No	Sí

Un usuario con rol “viewer” no puede ejecutar acciones de administrador: ni el botón debe estar visible, ni el endpoint debe responder a una llamada directa. Sin sesión, ninguna ruta ni endpoint de datos es accesible. Las credenciales del usuario admin se **siembran desde variables de entorno** (nunca escritas en el código).

Duración y modalidad

- **Duración:** 8 horas, individual.
- Puedes consultar documentación oficial (Airflow, USGS, Docker, PostgreSQL, FastAPI, pytest, etc.) libremente durante la prueba.
- Se evalúa el manejo correcto de los temas, no la cantidad de DAGs, pantallas o gráficos.
- **Recomendación:** prioriza tener un flujo completo funcionando de punta a punta (fuente → Airflow → base de datos → plataforma) antes de perfeccionar una sola capa.

Stack y herramientas

- **Orquestación (obligatorio):** Apache Airflow, levantado con Docker Compose.
- **Lenguaje del pipeline:** Python 3.11 o superior.
- **Almacenamiento:** PostgreSQL (contenedor local o Supabase) con schemas separados `raw`, `staging` y `curated`.
- **Calidad de datos:** Great Expectations, Pandera, Soda, dbt tests o validaciones propias en SQL/Python; decide cuál usar y justifícalo en el README.
- **IA:** proveedor de LLM a tu elección (Claude, OpenAI, Gemini o un modelo local con Ollama) más una técnica analítica de detección de anomalías.
- **Plataforma de consumo:** backend API (se recomienda FastAPI) y frontend a tu criterio (React + TypeScript + Vite, Streamlit, Dash u otro). Herramientas de BI como Grafana o Metabase pueden complementar, pero **no reemplazan** la plataforma propia.
- **Seguridad:** al menos una herramienta de escaneo (gitleaks, trufflehog, bandit o pip-audit) con evidencia en el README.
- **Testing:** pytest.
- **Control de versiones:** Git, con commits descriptivos.

Puesta en marcha

- Crea tu propio repositorio. Todo el sistema debe levantar con un único comando: `docker compose up -d`.
- Configura tus secretos copiando `.env.example` a `.env`. El `.env` real **nunca** se sube al repositorio.
- Puertos sugeridos: Airflow en `http://localhost:8080`, API en `http://localhost:8000/docs` y plataforma web en `http://localhost:3000`.

- Cambia las credenciales por defecto de Airflow y genera tu propia clave Fernet.

Funcionalidades principales

Para alcanzar un resultado óptimo de la solución, deberás cumplir con los siguientes requisitos.

Requisitos

1. Arquitectura y propósito

- Documento `docs/ARQUITECTURA.md` que explique: el **propósito** de la arquitectura (qué problema resuelve, para quién y qué decisiones habilita), un **diagrama** de componentes y capas, y la justificación de cada herramienta elegida.
- Capas de datos claramente separadas: **raw** (dato tal como llega), **staging** (limpio y tipado) y **curated** (listo para consumo).
- Modelo de datos documentado (tablas, llaves y relaciones).
- Sección de decisiones y trade-offs: por qué Airflow y no un cron, por qué esas capas, qué harías diferente en producción.

2. Ingesta y orquestación en Airflow

- **DAG de ingesta:** consulta el feed de USGS cada 5 minutos y guarda la respuesta en `raw`.
- **DAG de transformación y calidad:** limpia y tipa los datos hacia `staging` y publica en `curated` solo lo que pasa las validaciones; se dispara al terminar la ingesta (Dataset/Asset o `TriggerDagRun`).
- **DAG diario de reporte con IA** (ver punto 5).
- **Backfill histórico:** una ejecución parametrizada que cargue los últimos 30 días desde el endpoint de consulta, sin duplicar datos ya existentes.
- Cargas **idempotentes**: ejecutar dos veces el mismo intervalo no duplica registros (upsert por `id`).
- Reintentos con espera creciente, `execution_timeout`, `max_active_runs` controlado y manejo explícito del caso en que la API de USGS no responda.
- Alertas ante fallos mediante `on_failure_callback` (correo, Slack, Telegram o webhook, a tu elección).
- Buenas prácticas: TaskFlow API o tareas bien separadas, TaskGroups, tags, `doc_md` en cada DAG, sin lógica pesada en el archivo del DAG y sin enviar datos grandes por XCom.

3. Calidad de datos

- Reglas mínimas: esquema y tipos esperados, campos obligatorios (`id`, `time`, coordenadas), rangos válidos (latitud −90 a 90, longitud −180 a 180, magnitud y profundidad plausibles), unicidad de `id`, detección de duplicados y formato de fecha.

- Controles de **frescura** (la última carga exitosa no supera un umbral definido) y de **volumen** (conteo de eventos anormalmente bajo o alto).
- Las validaciones son **tareas bloqueantes** en Airflow: si fallan, los datos no llegan a `curated`.
- Los registros inválidos no se pierden: van a una tabla de **cuarentena** con el motivo del rechazo.
- Cada ejecución guarda sus métricas en una tabla de resultados de calidad (reglas evaluadas, aprobadas, rechazadas y puntaje) y esa información se muestra en la plataforma.

4. Seguridad

- **Gestión de secretos:** ninguna credencial ni API key en el código ni en el repositorio; uso de `.env`, Airflow Connections/Variables o un secrets backend; `.gitignore` correcto.
- **Mínimo privilegio en la base de datos:** roles distintos para ingesta, transformación y lectura; la plataforma solo lee `curated`.
- **Plataforma:** autenticación con JWT, control de acceso por rol, validación de entradas, CORS restringido y sin exponer `raw` ni la base de datos directamente.
- **Datos externos no confiables:** HTTPS y timeouts en las llamadas, validación del payload y sanitización de textos (por ejemplo `place`) antes de enviarlos a un LLM, para mitigar inyección de instrucciones (prompt injection).
- Al menos un **escaneo de seguridad** ejecutado (secretos, dependencias o código) con su resultado documentado.
- Logs sin secretos ni datos sensibles.

5. Automatización con IA

Debes implementar **al menos dos automatizaciones con IA**, integradas al orquestamiento:

- **Reporte narrativo (IA generativa):** un DAG genera cada día un resumen ejecutivo en lenguaje natural (qué pasó, zonas más activas, eventos destacados) a partir de los datos de `curated`.
- **Detección de anomalías (analítica/ML):** identifica comportamientos inusuales (por ejemplo, enjambres sísmicos o picos de actividad) con una técnica a tu elección (z-score, IsolationForest, DBSCAN espacial, etc.) y genera una explicación breve de cada alerta.

Controles obligatorios:

- **Las cifras las calcula el código (SQL/Python), no el LLM.** El modelo solo redacta; debes validar que los números del texto coincidan con los calculados.
- Salida estructurada y validada (por ejemplo JSON con esquema Pydantic) y un **plan alterno** si el proveedor de IA falla o excede el tiempo.
- Prompts versionados en el repositorio y registro de cada ejecución de IA (modelo, tokens, latencia y costo estimado) en una tabla.

- **Bono opcional:** consultas en lenguaje natural sobre los datos, con generación de SQL de solo lectura y restringida a `curated`.

6. Plataforma de consumo de datos

Una aplicación web propia, con backend que lee únicamente de `curated`:

- **Dashboard:** KPIs (eventos en 24 h, magnitud máxima, magnitud promedio, eventos con alerta de tsunami, puntaje de calidad y hora de última actualización), gráficos (serie temporal por hora, distribución por magnitud, regiones con más actividad) y **mapa** de eventos. Filtros por rango de fechas, magnitud mínima y región.
- **Reportes:** historial de reportes diarios generados por IA, tabla paginada con búsqueda y orden, y **exportación** a CSV y a PDF o HTML descargable.
- **Tiempo real:** los eventos nuevos aparecen **sin recargar la página** mediante WebSocket o Server-Sent Events (si usas polling, que sea corto, de máximo 30 segundos, y justifícalo). Debe mostrar un indicador “en vivo”, la hora de la última actualización y reconectar si se pierde la conexión.
- **Estado del pipeline:** última ejecución de cada DAG, resultado de calidad y frescura de los datos.
- Login con roles según el modelo de la sección anterior.
- Estados de carga, vacío y error en cada vista: nunca una pantalla en blanco.

7. Pruebas

- Una prueba unitaria de una función de transformación (por ejemplo, convertir un evento GeoJSON a una fila de `staging`).
- Una prueba de integridad de DAGs: todos los DAGs se importan sin errores.
- Una prueba de una regla de calidad (un caso válido y uno inválido).
- Una prueba de un endpoint de la API, incluida la verificación de que un rol sin permiso recibe 401/403.

Endpoints mínimos sugeridos para tu plataforma

El diseño de la API es libre; esta tabla es una referencia del alcance esperado.

Método	Ruta	Rol	Descripción
POST	/auth/login	—	Inicio de sesión, devuelve JWT.
GET	/api/kpis	viewer, admin	Indicadores del dashboard.
GET	/api/events	viewer, admin	Eventos con filtros y paginación.
GET	/api/stats/timeseries	viewer, admin	Serie temporal para gráficos.
GET	/api/reports	viewer, admin	Historial de reportes (incluye los de IA).
GET	/api/reports/export	viewer, admin	Exportación CSV / PDF.

Método	Ruta	Rol	Descripción
GET	/api/stream	viewer, admin	Eventos en tiempo real (SSE) o WebSocket /ws.
GET	/api/pipeline/status	viewer, admin	Estado de DAGs y calidad.
GET	/api/quality/quarantine	admin	Registros rechazados por calidad.
POST	/api/admin/backfill	admin	Lanza un backfill histórico.
POST	/api/admin/reports/regenerate	admin	Regenera el reporte con IA.

Criterios de aceptación

- `docker compose up -d` levanta Airflow, la base de datos y la plataforma; los DAGs aparecen sin errores en la interfaz de Airflow.
- Los datos fluyen de forma automática desde USGS hasta el dashboard, sin ejecutar ningún paso manual.
- Ejecutar un DAG dos veces sobre el mismo intervalo no genera registros duplicados.
- Un dato inválido nunca llega a `curated`: queda en cuarentena con el motivo y se refleja en las métricas de calidad.
- No existe ningún secreto en el repositorio ni en su historial; el escaneo de seguridad lo evidencia.
- Sin token, ninguna ruta ni endpoint de datos es accesible; un “viewer” que intenta una acción de admin recibe un rechazo claro.
- El reporte de IA se genera automáticamente y sus cifras coinciden con las calculadas por el código; si el proveedor de IA falla, el pipeline no se rompe.
- Los eventos nuevos se ven en la plataforma en tiempo real sin recargar la página.
- Ningún fallo de la API, de la base de datos o de la conexión en vivo deja al usuario ante una pantalla en blanco.
- Las pruebas corren con un solo comando (`pytest`) y pasan.

Entregable

Archivo .zip del proyecto subido a la plataforma Moodle a tiempo, más el repositorio con el proyecto completo y un `README.md` que explique:

- Cómo levantar y correr el proyecto localmente (incluyendo variables de entorno necesarias).
- El **propósito de la arquitectura**, el diagrama y las decisiones principales (puede apoyarse en `docs/ARQUITECTURA.md`).
- Cómo implementaste la calidad de datos: herramienta elegida, reglas y dónde se ven los resultados.
- Cómo resolviste la seguridad: manejo de secretos, roles de base de datos, autenticación y evidencia del escaneo.

- Cómo implementaste la IA: proveedor, prompts, controles contra información inventada y manejo de fallos.
- Cómo funciona el tiempo real (WebSocket, SSE o polling) y por qué lo elegiste.
- Capturas del dashboard, del reporte y de Airflow con los DAGs ejecutados.
- Limitaciones conocidas y mejoras que harías con más tiempo.

Estructura de repositorio sugerida:

```
.
├── docker-compose.yml
├── .env.example
├── README.md
├── docs/ARQUITECTURA.md
├── airflow/
│   ├── dags/
│   ├── include/            # transformaciones, calidad, cliente de IA, prompts
│   └── tests/
├── db/init/                # schemas, roles y tablas
├── api/                    # backend de la plataforma
└── web/                    # frontend de la plataforma
```

Consideraciones Generales

- Se permite el uso de herramientas de Inteligencia Artificial (asistentes, copilotos de programación) haciendo buen uso de ellas. **Eres el responsable de entender, revisar y poder explicar cada línea que entregas**; no se acepta entregar código que no puedas defender en la sustentación.
- Se permite la reutilización de lógica, componentes y código propio desarrollados en proyectos realizados durante las sesiones de entrenamiento.
- Está prohibido compartir el enunciado, los archivos o cualquier contenido relacionado con la prueba de desempeño con otros coders o terceros.
- Durante la ejecución de la prueba no está permitido consultar a otros coders sobre aspectos técnicos, solicitar ayuda para resolver ejercicios o discutir cualquier tema relacionado con la evaluación.
- La prueba se cerrará automáticamente a las 13:59.
- Las entregas realizadas después de la hora de cierre serán recibidas con una penalización de 10 puntos sobre la calificación final obtenida.
- El coder es responsable de verificar que la entrega se haya realizado correctamente en la plataforma Moodle.

Recursos

- Apache Airflow: https://airflow.apache.org/docs/
- Airflow con Docker Compose: https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html
- USGS Earthquake API: https://earthquake.usgs.gov/fdsnws/event/1/
- Great Expectations: https://docs.greatexpectations.io
- Pandera: https://pandera.readthedocs.io

- FastAPI: https://fastapi.tiangolo.com
- Docker Compose: https://docs.docker.com/compose/
- pytest: https://docs.pytest.org
- OWASP Top 10 para aplicaciones con LLM: https://owasp.org/www-project-top-10-for-large-language-model-applications/

Be a coder